

0.1 Giới thiệu chương

Chương này trình bày những giải pháp và đóng góp nổi bật mà em đã xây dựng trong quá trình thực hiện đề tài hệ thống giám sát và bảo vệ xe. Thay vì mô tả lại toàn bộ chức năng đã được giới thiệu ở các chương trước, nội dung chương tập trung vào các vấn đề có tính kỹ thuật, những khó khăn phát sinh trong quá trình thiết kế, các phương án đã được cân nhắc, giải pháp được lựa chọn và kết quả đạt được.

Bốn đóng góp chính được trình bày gồm: giải thuật phát hiện xe di chuyển bất thường; cơ chế cảnh báo đa kênh kết hợp Socket.IO và Firebase Cloud Messaging; phương pháp tái tạo hành trình và tính quãng đường từ dữ liệu GPS; giải pháp quản lý vòng đời và quyền điều khiển thiết bị IoT. Đây là các nội dung thể hiện rõ nhất quá trình phân tích, giải quyết vấn đề và hoàn thiện hệ thống của tác giả.

0.2 Giải thuật phát hiện xe di chuyển bất thường

0.2.1 Bài toán và khó khăn

Mục tiêu quan trọng nhất của hệ thống là phát hiện trường hợp xe bị di chuyển khi người dùng đã bật chế độ bảo vệ. Nếu chỉ kiểm tra xem xe có tốc độ lớn hơn không, hệ thống có thể bỏ sót trường hợp dữ liệu tốc độ chưa được cập nhật nhưng tọa độ GPS đã thay đổi. Ngược lại, nếu chỉ dựa trên khoảng cách giữa hai tọa độ, sai số tự nhiên của GPS có thể làm hệ thống hiểu nhầm rằng xe đang di chuyển dù xe vẫn đứng yên.

Dữ liệu GPS còn có thể xuất hiện các điểm trôi, điểm nhảy hoặc bản tin không ổn định do chất lượng tín hiệu vệ tinh. Một bản tin bất thường đơn lẻ vì vậy chưa đủ độ tin cậy để kết luận xe đang gặp nguy hiểm. Việc phát cảnh báo ngay từ bản tin đầu tiên sẽ làm tăng tỷ lệ cảnh báo giả, gây khó chịu và khiến người dùng dần bỏ qua các cảnh báo thật.

Ngoài ra, cùng một dữ liệu tốc độ và vị trí phải được xử lý khác nhau tùy theo trạng thái của xe. Khi người dùng tắt bảo vệ, xe di chuyển là hành vi bình thường. Khi người dùng bật bảo vệ, vị trí tại thời điểm đó trở thành vị trí tham chiếu và mọi dịch chuyển đáng kể sau đó cần được kiểm tra. Vì vậy, bài toán cần kết hợp trạng thái bảo vệ, tốc độ, khoảng cách và lịch sử các bản tin liên tiếp.

0.2.2 Giải pháp đề xuất

Khi chế độ bảo vệ được bật, hệ thống ghi nhận vị trí gần nhất của xe làm vị trí tham chiếu. Với mỗi bản tin MQTT mới, backend chuẩn hóa tốc độ và tọa độ trước khi đánh giá rủi ro. Khoảng cách giữa vị trí hiện tại và vị trí bảo vệ được tính bằng công thức Haversine.

Gọi φ_1, φ_2 là vĩ độ; λ_1, λ_2 là kinh độ của hai điểm tính theo radian. Khoảng cách được xác định như sau:

$$a = \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos(\varphi_1) \cos(\varphi_2) \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (1)$$

$$d = 2R \arcsin(\sqrt{a}) \quad (2)$$

Trong đó, $R = 6.371.000$ m là bán kính trung bình của Trái Đất. Hệ thống sử dụng các điều kiện đánh giá sau: Bất thường tốc độ được xác định khi tốc độ lớn hơn 10 km/h. Bất thường khoảng cách được xác định khi khoảng cách di chuyển lớn hơn 50 m. Trường hợp di chuyển nghiêm trọng xảy ra khi khoảng cách di chuyển lớn hơn 100 m. Một bản ghi được coi là dữ liệu nghi ngờ khi xuất hiện bất thường tốc độ hoặc bất thường khoảng cách.

Để giảm cảnh báo giả, biến `suspiciousCount` lưu số bản tin bất thường liên tiếp. Bản tin bất thường đầu tiên chỉ làm tăng bộ đếm. Cảnh báo được tạo khi hệ thống nhận đủ hai bản tin bất thường liên tiếp. Nếu dữ liệu trở lại bình thường hoặc người dùng tắt chế độ bảo vệ, bộ đếm được đặt lại. Biến `lastSpeedAlert` được sử dụng để ngăn gửi nhiều cảnh báo cho cùng một đợt bất thường.

Xe được đánh dấu ở trạng thái nguy hiểm khi khoảng cách vượt 100 m hoặc đồng thời xuất hiện cả bất thường về tốc độ và khoảng cách. Quy trình xử lý tổng quát được mô tả như sau:

- Nhận dữ liệu từ thiết bị thông qua giao thức MQTT.
- Kiểm tra thông tin thiết bị và trạng thái kích hoạt.
- Lưu dữ liệu cảm biến vào cơ sở dữ liệu.
- Kiểm tra chế độ bảo vệ của phương tiện.
- Chuẩn hóa tốc độ và tọa độ.
- Tính khoảng cách di chuyển bằng công thức Haversine.
- Kiểm tra các dấu hiệu bất thường.
- Cập nhật bộ đếm bất thường liên tiếp.
- Tạo cảnh báo nếu thỏa mãn điều kiện cảnh báo.

Giải pháp này có thể tổng quát hóa cho các hệ thống giám sát đối tượng di động khác bằng cách thay đổi ngưỡng tốc độ, khoảng cách, số mẫu liên tiếp và quy tắc phân loại mức độ nghiêm trọng.

0.2.3 Kết quả đạt được và đánh giá

Giải thuật đã được tích hợp vào luồng tiếp nhận dữ liệu MQTT của backend. Dữ liệu cảm biến được lưu trước khi đánh giá rủi ro, nhờ đó có thể đối chiếu lại lịch sử khi cần. Khi đủ điều kiện, hệ thống tạo bản ghi cảnh báo, cập nhật trạng thái thiết bị và chuyển thông tin đến ứng dụng.

Tình huống	Kết quả
Tốc độ không vượt 10 km/h và khoảng cách không vượt 50 m	Không tạo cảnh báo
Chỉ có một bản tin vượt ngưỡng	Tăng bộ đếm, chưa tạo cảnh báo
Hai bản tin liên tiếp vượt ngưỡng	Tạo cảnh báo mới
Khoảng cách vượt 100 m	Tạo cảnh báo khẩn cấp và đánh dấu nguy hiểm
Đồng thời vượt tốc độ và khoảng cách	Tạo cảnh báo khẩn cấp
Dữ liệu trở lại bình thường	Đặt lại bộ đếm bất thường

Bảng 1: Kết quả mong đợi của giải thuật phát hiện bất thường

Ưu điểm của giải pháp là không phụ thuộc vào một nguồn thông tin duy nhất và có cơ chế chống cảnh báo lặp. Tuy nhiên, các ngưỡng hiện được lựa chọn dựa trên môi trường thử nghiệm và chưa được hiệu chỉnh bằng tập dữ liệu thực tế đủ lớn.

0.3 Cơ chế cảnh báo đa kênh

0.3.1 Bài toán và khó khăn

Ứng dụng cần thông báo cho người dùng trong nhiều trạng thái khác nhau. Khi ứng dụng đang mở và kết nối mạng ổn định, dữ liệu cần được cập nhật gần thời gian thực. Khi ứng dụng chạy nền hoặc đã bị hệ điều hành tạm dừng, kết nối Socket.IO có thể không còn hoạt động. Nếu chỉ sử dụng Socket.IO, người dùng có nguy cơ không nhận được cảnh báo quan trọng. Nếu chỉ sử dụng push notification, giao diện khi đang mở có thể cập nhật chậm và không tạo được cảm giác realtime.

Một khó khăn khác là dữ liệu chỉ được phép gửi đến đúng chủ sở hữu thiết bị. Backend không thể phát toàn bộ cảnh báo cho tất cả kết nối đang hoạt động. Đồng thời, khi cảnh báo được nhận, trạng thái của xe trên trang chủ, danh sách thiết bị, màn hình chi tiết và danh sách cảnh báo phải được đồng bộ mà không yêu cầu người dùng tải lại trang.

0.3.2 Giải pháp đề xuất

Hệ thống sử dụng hai kênh cảnh báo bổ trợ cho nhau. Socket.IO phục vụ cập nhật realtime khi ứng dụng đang hoạt động. Firebase Cloud Messaging đóng vai trò kênh thông báo khi ứng dụng chạy nền, bị tạm dừng hoặc kết nối Socket.IO không sẵn sàng.

Quá trình gửi cảnh báo được thực hiện bởi ‘SensorDataService’ gồm các bước sau:

- Lưu thông tin cảnh báo vào cơ sở dữ liệu PostgreSQL.
- Gửi sự kiện cảnh báo thời gian thực thông qua ‘SensorDataGateway’ sử dụng Socket.IO.
- Gửi thông báo đẩy đến người dùng thông qua ‘FirebaseMessagingService’ và Firebase Cloud Messaging.

Mỗi client Socket.IO gửi JWT sau khi kết nối. Backend xác thực token, lấy `userId` và gán định danh người dùng với socket tương ứng. Khi phát sinh cảnh báo, gateway chỉ gửi sự kiện đến các socket có cùng `userId`. Đối với FCM, token của từng thiết bị di động được lưu trong bảng `FcmToken`. Backend lấy toàn bộ token hợp lệ của người dùng và gửi cảnh báo multicast.

Ở phía Flutter, sự kiện realtime không chỉ được chuyển cho bộ điều khiển cảnh báo mà còn được chuyển cho bộ điều khiển thiết bị. Trường `vehicleStatus` trong payload được dùng để cập nhật ngay trạng thái `Stolen`. Vì vậy, trang chủ, danh sách thiết bị và màn hình chi tiết cùng thay đổi mà không cần gọi lại API. Khi người dùng mở ứng dụng từ FCM, hệ thống tải lại thiết bị và cảnh báo để đồng bộ với database.

Thông báo vẫn được nhận dù người dùng đang lọc danh sách theo ngày hoặc thiết bị. Bộ lọc chỉ giới hạn dữ liệu hiển thị trong danh sách, không ngăn local notification hoặc push notification khẩn cấp.

0.3.3 Kết quả đạt được và đánh giá

Giải pháp giúp ứng dụng hoạt động phù hợp ở cả foreground và background. Khi ứng dụng đang mở, Socket.IO cập nhật giao diện trực tiếp. Khi ứng dụng chạy nền, FCM tạo thông báo hệ thống và điều hướng người dùng đến màn hình cảnh báo khi được nhấn.

Backend đồng thời loại bỏ các FCM token đã hết hiệu lực dựa trên phản hồi của Firebase. Điều này hạn chế việc tiếp tục gửi đến token không còn tồn tại. Cảnh báo được lưu vào database trước khi gửi qua hai kênh, nhờ đó người dùng vẫn có thể xem lại lịch sử nếu việc gửi realtime hoặc push gặp sự cố.

Hạn chế của giải pháp là chưa xây dựng cơ chế hàng đợi và thử gửi lại khi dịch vụ ngoài tạm thời không khả dụng. Hệ thống hiện cũng chưa lưu trạng thái đã đọc của từng cảnh báo. Các nội dung này có thể được phát triển bằng message queue, retry có giới hạn và trường `readAt` trong bảng cảnh báo.

0.4 Tái tạo hành trình và tính quãng đường từ dữ liệu GPS

0.4.1 Bài toán và khó khăn

Dữ liệu GPS được thiết bị gửi theo thời gian, nhưng không phải mọi điểm đều có thể sử dụng trực tiếp để vẽ hành trình. Tọa độ có thể sử dụng các tên trường khác nhau như `lat`, `latitude`, `lng` hoặc `longitude`. Một số bản ghi có thể thiếu tọa độ, vượt phạm vi hợp lệ, trùng nhau hoặc xuất hiện bước nhảy lớn do mất tín hiệu.

Nếu cộng khoảng cách giữa mọi cặp điểm mà không lọc, sai số vài mét khi xe đứng yên sẽ tích lũy thành một quãng đường đáng kể. Bước nhảy GPS cũng có thể làm tổng quãng đường tăng hàng kilomet dù xe chỉ di chuyển trong phạm vi nhỏ. Vì vậy, chức năng này cần vừa tái tạo đúng thứ tự hành trình vừa hạn chế nhiễu.

0.4.2 Giải pháp đề xuất

Backend nhận khoảng thời gian cần truy vấn, lấy các bản ghi cảm biến theo thứ tự tăng dần của `createdAt` và chuẩn hóa tọa độ về cấu trúc thống nhất gồm `lat` và `lng`. Các điểm nằm ngoài phạm vi vĩ độ $[-90, 90]$ hoặc kinh độ $[-180, 180]$ bị loại bỏ.

Khoảng cách giữa điểm mới và điểm hợp lệ gần nhất được tính bằng Haversine. Một điểm bị bỏ qua nếu khoảng cách nhỏ hơn 3 m vì nhiều khả năng đây là rung GPS khi xe đứng yên. Hệ thống cũng tính vận tốc suy ra từ khoảng cách và chênh lệch thời gian. Nếu vận tốc suy ra vượt 200 km/h, điểm mới được xem là bước nhảy không hợp lý và không được cộng vào tổng quãng đường.

Quy trình tính toán quãng đường di chuyển của thiết bị gồm các bước sau:

- Lấy danh sách các điểm dữ liệu theo thứ tự thời gian tăng dần.
- Chuẩn hóa tọa độ của từng điểm dữ liệu.
- Loại bỏ các tọa độ không hợp lệ.
- Loại bỏ các điểm có mức dịch chuyển nhỏ hơn 3 m để giảm nhiễu GPS.
- Loại bỏ các điểm có vận tốc suy ra lớn hơn 200 km/h do có khả năng là dữ liệu sai lệch.
- Tính và cộng dồn khoảng cách giữa các điểm liên tiếp bằng công thức Haversine.
- Trả về danh sách các điểm hợp lệ và tổng quãng đường di chuyển.

App sẽ chuyển các điểm hợp lệ thành danh sách `LatLng` và sử dụng `PolylineLayer` để vẽ hành trình trên OpenStreetMap. Điểm đầu, điểm cuối, số điểm

GPS và tổng quãng đường được hiển thị trực tiếp. Người dùng có thể chọn phạm vi 24 giờ, 7 ngày hoặc toàn bộ lịch sử.

0.4.3 Kết quả đạt được và đánh giá

Chức năng đã cho phép người dùng quan sát tuyến đường của xe thay vì chỉ xem vị trí cuối cùng. Tổng quãng đường được tính ở backend nên kết quả không phụ thuộc vào khả năng xử lý của điện thoại. Việc lọc theo thời gian cũng làm giảm lượng dữ liệu truyền đến ứng dụng.

Các quy tắc 3 m và 200 km/h giúp loại bỏ các trường hợp nhiễu rõ ràng, nhưng chưa thay thế được các thuật toán làm mượt GPS chuyên dụng. Khi có dữ liệu thực tế, có thể đánh giá thêm bộ lọc Kalman, map matching hoặc sử dụng độ chính xác do module GPS cung cấp. Đây là hướng tổng quát hóa giải pháp cho các hệ thống theo dõi phương tiện có yêu cầu độ chính xác cao hơn.

0.5 Quản lý vòng đời và quyền điều khiển thiết bị IoT

0.5.1 Bài toán và khó khăn

Thiết bị IoT được tạo trước khi thuộc về một tài khoản cụ thể. Nếu chỉ sử dụng `deviceId`, một người biết mã thiết bị có thể kích hoạt thiết bị không thuộc sở hữu của mình. Hệ thống cũng phải ngăn người dùng xem dữ liệu, thay đổi trạng thái bảo vệ hoặc điều khiển còi của thiết bị thuộc người dùng khác.

Ở chiều MQTT, backend nhận dữ liệu từ broker thay vì nhận trực tiếp từ một kết nối HTTP đã có JWT. Do đó cần một cơ chế riêng để xác thực nguồn gửi dữ liệu mà không làm phát sinh truy vấn database cho mọi bản tin cảm biến.

0.5.2 Giải pháp đề xuất

Mỗi thiết bị được định danh bằng `deviceId` và một khóa bí mật `deviceKey`. Khi kích hoạt, người dùng phải đăng nhập và cung cấp đúng cả hai giá trị. Backend kiểm tra thiết bị chưa thuộc người dùng khác, sau đó cập nhật `userId`, `isActive` và `activatedAt`.

Các API dành cho người dùng đều kiểm tra JWT. Sau khi tìm thiết bị, service so sánh `device.userId` với `req.user.id`. Yêu cầu không đúng chủ sở hữu bị từ chối trước khi truy vấn dữ liệu hoặc gửi lệnh MQTT.

Thiết bị gửi `deviceKey` trong payload MQTT. Kết quả xác thực thành công được lưu trong cache một giờ. Các bản tin tiếp theo của cùng thiết bị không cần truy vấn database lại trong thời gian cache còn hiệu lực.

Luồng điều khiển còi cũng kiểm tra quyền sở hữu trước khi publish lệnh đến topic `buzzer/deviceId`. Trạng thái còi trong database chỉ được cập nhật sau

khi yêu cầu đã vượt qua bước kiểm tra quyền.

0.5.3 Kết quả đạt được và đánh giá

Giải pháp bảo đảm một thiết bị không thể được kích hoạt chỉ bằng mã công khai, không thể đồng thời thuộc nhiều người dùng và không thể bị điều khiển bởi tài khoản không có quyền. Cache xác thực làm giảm số truy vấn database khi thiết bị gửi dữ liệu với tần suất cao.

Tuy vậy, `deviceKey` hiện vẫn là khóa tĩnh. Firmware thử nghiệm còn chứa thông tin cấu hình trực tiếp và sử dụng kết nối chưa kiểm tra đầy đủ chứng thư. Khi triển khai thực tế, khóa cần được lưu trong vùng bảo mật, hỗ trợ xoay vòng, không ghi vào mã nguồn và kết nối MQTT cần xác minh chứng thư máy chủ.

0.6 Tổng kết chương

Chương đã trình bày bốn đóng góp tập trung vào các vấn đề cốt lõi của hệ thống giám sát và bảo vệ xe. Giải thuật phát hiện bất thường kết hợp trạng thái bảo vệ, tốc độ, khoảng cách và chuỗi bản tin liên tiếp để giảm cảnh báo giả. Cơ chế cảnh báo đa kênh giúp hệ thống hoạt động ở cả foreground và background, đồng thời đồng bộ trạng thái giao diện theo thời gian thực. Phương pháp xử lý GPS cung cấp hành trình và quãng đường có lọc nhiễu cơ bản. Giải pháp kích hoạt và phân quyền thiết bị tạo ra ranh giới rõ ràng giữa người dùng, thiết bị và kênh MQTT.